ResearchMind — Project Deep Dive & Interview Prep

1. The 30-second pitch

A Streamlit app where four LLM-powered roles collaborate sequentially: a **Search Agent** finds sources on the web (Tavily), a **Reader Agent** scrapes the most relevant page for depth, a **Writer** turns both into a structured report, and a **Critic** scores/reviews that report. Built with LangChain's agent framework (`create_agent`, LangGraph-backed) for the tool-using steps, plain LCEL chains for the deterministic writing/reviewing steps, and Mistral as the LLM throughout.

2. The four-stage pipeline (your architecture answer)

```
1. SEARCH   build_search_agent()  ->  agent decides to call web_search(query)
-> Tavily results
2. READ     build_reader_agent()  ->  agent decides to call scrape_url(url)
-> page text
3. WRITE    writer_chain          ->  {topic, research} -> structured report
(fixed prompt, no tools)
4. CRITIQUE critic_chain          ->  {report} -> score + strengths + areas
to improve (fixed prompt)
```

Steps 1–2 are **agents** (the LLM chooses *whether and how* to call a tool). Steps 3–4 are **chains** (a fixed prompt → LLM → parser, no decision-making, no tools). Being able to state *why* each step is one or the other is one of the strongest things you can say in this interview — see Section 4.

---

3. Module-by-module walkthrough

3.1 `tools.py` — the two capabilities agents can call

- `web_search(query)` — wraps `TavilyClient.search()`, formats up to 5 results into a text block of title/URL/snippet. Tavily specifically (vs. a raw Google scrape) because it's built for LLM consumption — it returns clean structured snippets rather than raw SERP HTML you'd have to parse yourself.
- `scrape_url(url)` — fetches a page with a browser-like `User-Agent` header (many sites block requests without one), strips `script`/`style`/`nav`/`footer`/`header`/`aside`/`noscript` tags with BeautifulSoup (these are boilerplate, not content), collapses whitespace, and **truncates to 3000 characters.**

Both are decorated with `@tool` from `langchain.tools` — this is what turns a plain Python function into something an LLM agent can discover and invoke: the function's docstring

becomes the tool's description the LLM uses to decide *when* to call it, and the type hints define the expected argument schema.

Why truncate scraped content to 3000 chars? Same reasoning as chunk sizing in any LLM pipeline — cost and context-window control. A full article could be 10,000+ characters; the writer chain doesn't need the whole page, just enough substance to write from.

3.2 `agents.py` — where agents and chains are built

Four things happen in this file, and they're architecturally different from each other:

- **`build_search_agent()`** and **`build_reader_agent()`** — each calls `create_agent(model=llm, tools=[...], system_prompt="...")`. This is LangChain's higher-level agent constructor (backed by LangGraph under the hood) — it wires up a **tool-calling loop**: the LLM receives the system prompt + user message, decides whether to call a tool, the tool result is fed back to the LLM as an observation, and this repeats until the LLM produces a final answer instead of another tool call. The system prompts explicitly say *"Always use the X tool. Never answer from your own knowledge."* — that instruction is doing real work: it forces tool use rather than letting the model guess from training data, which is the whole point of a search agent.
- **`writer_chain`** and **`critic_chain`** — plain LCEL (`prompt | llm | StrOutputParser()`), no tools, no decision loop. They take structured input (already-gathered research, or an already-written report) and transform it deterministically. There's nothing for them to *decide* — they're transformation steps, not research steps — so an agent would be over- engineering; a chain is the right-sized tool.

⚠️ **There's a real bug in this file, worth knowing cold if asked to review it:**

```python
if "MISTRAL_API_KEY" in st.secrets:
```

`agents.py` never imports `streamlit as st` — only `app.py`/`test.py` do. This line references an undefined name `st` and will raise `NameError` the moment `agents.py` is imported, which means **the app cannot start at all** in its current form. Related issues in the same block:

- `api_key` is checked (`if not api_key`) and passed to `ChatMistralAI(api_key=api_key, ...)`, but the variable that was actually assigned above it is `mistral_key`, not `api_key` — a second `NameError` waiting right behind the first.
- `llm` is defined **twice** — once inside the broken secrets/env block, once again unconditionally a few lines later (`llm = ChatMistralAI(model=..., temperature=0)`), relying on `langchain-mistralai` picking up `MISTRAL_API_KEY` from the environment automatically). The second definition silently overwrites the first, making the whole secrets-vs-.env branching logic dead code even if the `NameError`s were fixed.

- `st.title("Mistral AI App")` is called from `agents.py` — a business-logic file rendering UI. That's a layering violation: `agents.py` should build agents/chains, `app.py` should own all Streamlit calls. If this line ever executed outside a Streamlit script context it would also error.

This is genuinely good interview material: being able to say *"here's a bug I found, here's why it fails, here's how I'd fix it"* (move the secrets-vs-.env check into `app.py`, delete the duplicate `llm` definition, delete the stray `st.title` call) demonstrates real code-reading skill, not just recall.

3.3 `pipeline.py` vs `app.py` — two orchestrators, inconsistent

Both call the same four steps in the same order, but unlike your previous project (where `main.py`/`app.py` both called into one clean shared pipeline), here the logic is **duplicated, not shared**, and it's drifted:

- `pipeline.py`'s `run_research_pipeline(topic)` truncates the search results to **800 characters** before handing them to the reader agent (`state['search_results'][:800]`).
- `app.py` passes the **full, untruncated** search results to the reader agent.

That's a real inconsistency worth flagging if asked "what would you clean up first" — the CLI and the web UI currently produce pipelines that behave differently, purely because the same four-step logic was written twice instead of `app.py` importing and calling `run_research_pipeline` from `pipeline.py`.

3.4 `test.py` — not actually a test

Despite the name, `test.py` is a byte-for-byte duplicate of `app.py` — same Streamlit page, same CSS, same logic. No `assert`s, no `pytest`/`unittest` usage. Two things worth being able to say about this:

1. It's presumably a leftover working copy, not a real test suite — good to be upfront about that if asked, rather than claiming it's a test file.
2. Naming it `test.py` risks colliding with `pytest`'s auto-discovery (`test_*.py` or `*_test.py` patterns) if a real test suite is ever added to this project — worth renaming to avoid confusion once actual tests exist.

3.5 `app.py` — the Streamlit orchestrator

Structurally similar to your video-assistant app: `st.session_state` holds `results`/`running`/`finished`/`topic` so state survives Streamlit's rerun-on- interaction model. The interesting piece is `render_pipeline(active_idx)` — a single function that renders all four step-cards with a `waiting`/`running`/ `done` state each, driven by one integer (`active_idx`). That's a clean way to avoid four near-identical blocks of conditional UI code — worth pointing out as a deliberate simplification if asked about UI structure.

`text_of = lambda x: x.content if hasattr(x, "content") else str(x)` — a small but important normalization: LangChain message objects sometimes come back as `AIMessage` objects (with `.content`) and sometimes as plain strings depending on which call produced them; this lambda makes the rest of the UI code not care which one it got.

---

4. The concepts underneath (know these cold)

Agent vs. Chain — the single most important distinction in this project. A *chain* is a fixed sequence of steps you wrote (`prompt | llm | parser`) — the LLM has no control over what happens next. An *agent* gives the LLM a set of tools and lets **the model itself decide**, at runtime, whether to call a tool, which one, with what arguments, and when it has enough information to stop and answer. Search and reading are agentic here because the "right" URL to scrape, or the right search query, genuinely depends on the topic — that's a decision, not a fixed transformation. Writing and critiquing are chains because, given the research, there's no decision left to make — just a transformation from input text to output text.

The tool-calling loop, concretely: system prompt + user message go to the LLM → LLM emits either a final text answer or a "call this tool with these args" instruction → if it's a tool call, the tool function actually executes in your Python process, its return value gets appended to the conversation as an "observation" → the LLM sees that and decides again → repeat until it emits a final answer instead of a tool call. `create_agent` (LangGraph under the hood) manages this loop for you — you don't write the "if tool call, execute and loop back" logic yourself.

Why force tool use via the system prompt ("Always use the web_search tool... Never answer from your own knowledge")? Without that instruction, a capable LLM might just answer a well-known topic from its training data and skip the tool entirely — defeating the purpose of a *research* agent, which is supposed to return current, sourced information, not the model's memorized priors.

Sequential multi-agent orchestration (this pattern, not LangGraph's graph- based orchestration): this project chains four independent agent/chain calls in a fixed order by passing the output of each as input to the next, entirely in plain Python (`pipeline.py`/`app.py`) — it does **not** use LangGraph's own graph/state-machine orchestration between the four steps, only *within* each individual agent (via `create_agent`). Worth being precise about this if asked "is this a LangGraph app" — LangGraph is used *inside* two of the four steps, not to orchestrate the four steps themselves.

Why Tavily instead of scraping a search engine directly? Search engines actively block scraping and return raw HTML you'd have to parse; Tavily is an API built specifically to return LLM-ready structured results (title/URL/ snippet), which is exactly the shape `web_search`'s docstring promises the agent.

---

5. Likely interview questions + how to answer them

"Walk me through what happens when a user submits a topic." → Use the 4-stage pipeline from Section 2, and explicitly name which two steps are agents and which two are chains, and why (Section 4).

"What's the difference between an agent and a chain in this codebase, and how do you decide which to use?" → Section 4's core answer. Good litmus test to state: *"if the right action depends on runtime information the model has to reason about, use an agent; if the transformation is fixed regardless of input, use a chain."*

"Walk me through `create_agent` — what's actually happening when `build_search_agent().invoke(...)` runs?" → The tool-calling loop from Section 4: LLM decides to call `web_search`, the function executes for real, results feed back as an observation, LLM either calls again or returns a final answer.

"If I asked you to review this code before merging, what would you flag?" → This is your strongest moment in this project — walk through the `agents.py` bug (Section 3.2) in order: undefined `st`, undefined `api_key`, duplicate `llm` definition, misplaced `st.title` call. Then the `pipeline.py`/`app.py` 800-char truncation inconsistency (Section 3.3). Naming real bugs you found yourself is far more convincing than reciting textbook LangChain concepts.

"How would you fix the Streamlit secrets vs .env logic?" → Move that branching entirely into `app.py` (which already imports streamlit), have it resolve the key and either pass it explicitly to a `build_search_agent(api_key=...)`-style factory or set `os.environ["MISTRAL_API_KEY"]` before `agents.py` is imported — `agents.py` itself shouldn't need to know whether the key came from `st.secrets` or `.env`.

"Why truncate scraped content to 3000 characters and search-result context to 800 in the pipeline version?" → Cost/context-window control, same reasoning as chunk sizes in any RAG/LLM pipeline — you want *enough* signal for the writer to work from, not the entire raw page. Flag the inconsistency (3000 in `tools.py`'s scraper itself, but a *second*, different truncation of the search results specifically in `pipeline.py` and not in `app.py`) as something you'd standardize.

"How would you make this more robust for production?" → Have 3–4 ready: (1) fix the `agents.py` crash bugs; (2) unify `pipeline.py` and `app.py` so there's one source of truth for the orchestration instead of copy-pasted, drifted logic; (3) add error handling around each `.invoke()` — right now if the search agent returns no usable message, the whole pipeline crashes with no user-facing explanation; (4) cache/reuse agent instances instead of rebuilding a fresh `create_agent(...)` graph on every single call.

"Why is the Critic step useful — what does it actually add?" → It's a self-review pattern (sometimes called an LLM-as-judge or reflection step): a second, independently-prompted pass over the same content catches gaps a single generation pass might not flag about itself,

and gives the user a transparent quality signal (score + strengths + areas to improve) rather than presenting the report as unreviewed ground truth.

---

6. If asked "what would you improve" — a ready list

1. Fix the `agents.py` crash bugs (undefined `st`, undefined `api_key`, duplicate `llm`, misplaced `st.title`).
2. Make `app.py` call `pipeline.py`'s `run_research_pipeline` instead of duplicating the four-step logic — one source of truth, no drift.
3. Standardize the search-result truncation (currently 800 chars in one entry point, untruncated in the other).
4. Add error handling around each agent/chain `.invoke()` call so one failed step doesn't crash the whole run with an unhandled exception.
5. Rename `test.py` to something like `app_v2_draft.py` (or delete it) so it doesn't collide with real pytest test discovery later, and add an actual test suite.
6. Cache built agents instead of reconstructing them on every pipeline run.
7. Let the Reader Agent scrape more than one URL (currently picks a single "most relevant" page) for a more thorough report — the README even lists "Multi-source research" as a planned improvement.